

JavaScript Revision Notes

Key Definitions

Hoisting

- JavaScript moves variable & function declarations to the top of their scope during compilation.
- `var` → hoisted as `undefined`.
- `let` / `const` → hoisted but not initialized (**Temporal Dead Zone**).

`var` / `let` / `const`

- `var` : function-scoped, can be re-declared.
- `let` : block-scoped, cannot be re-declared in same scope.
- `const` : block-scoped, must be initialized, cannot be reassigned.

`==` vs `===`

- `==` → loose equality (type coercion).
- `===` → strict equality (no coercion).

Shallow Copy vs Deep Copy

- **Shallow copy** → copies only first level, nested objects remain references.
- **Deep copy** → independent clone, no shared references.
- Methods: `...spread`, `slice`, `Object.assign` (shallow), `structuredClone`, `JSON.parse(JSON.stringify(obj))`, `_.cloneDeep` (deep).

Event Loop Execution Order

1. **Synchronous code** → executed immediately.
2. **Microtask queue** → high priority (`Promise.then`, `await`, `queueMicrotask`).
3. **Macrotask queue** → lower priority (`setTimeout`, `setInterval`, I/O).

 Rule: **Microtasks always run before the next macrotask.**

Event Loop Example

```
console.log("start");

setTimeout(() => console.log("timeout"), 0);

Promise.resolve().then(() => console.log("promise"));
```

```
console.log("end");
```

Output

```
start    // sync → printed immediately
end      // sync → printed immediately
promise  // microtask → runs before macrotask
timeout  // macrotask → runs last
```

Output-Based Questions & Explanations

Q1

```
console.log(typeof null);
console.log(null == undefined);
console.log(null === undefined);
```

Output

```
object    // typeof null is a legacy quirk
true      // null loosely equals undefined
false     // strict equality checks type too
```

Q2

```
var a = 1;

function test() {
  console.log(a);
  var a = 2;
  console.log(a);
}

test();
console.log(a);
```

Output

```
undefined // var a is hoisted, declared but not initialized
2         // local a is assigned after hoisting
1         // global a is untouched
```

Q3

```
let x = [1, 2, 3];
let y = x;
y.push(4);

console.log(x);
console.log(y);
console.log(x === y);
```

👉Output

```
[1,2,3,4] // x and y reference the same array
[1,2,3,4]
true      // both variables point to same object in memory
```

Q4

```
console.log([] + []);
console.log([] + {});
console.log({} + []);
console.log({} + {});
```

👉Output

```
""                // [] → "", so "" + "" → ""
"[object Object]" // [] → "", {} → string → "[object Object]"
0                // top-level {} is block, so +[] → 0
"[object Object][object Object]" // both objects coerced to strings
```

Q5

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => console.log("C"));

console.log("D");
```

👉Output

```
A // sync
D // sync
C // microtask runs before macrotask
B // macrotask runs last
```

Q6

```
async function test() {
  console.log("1");
  await Promise.resolve();
  console.log("2");
}

console.log("3");
test();
console.log("4");
```

👉Output

```
3 // sync
1 // sync part inside async before await
4 // sync after function call
2 // after await, resumes in microtask queue
```

Q7

```
console.log("A");

setTimeout(() => console.log("B"), 0);
```

```
(async function test() {  
  console.log("C");  
  await null;  
  console.log("D");  
})();  
  
console.log("E");
```

👉Output

```
A // sync  
C // sync  
E // sync  
D // microtask after await  
B // macrotask from setTimeout
```

Q8 (Tough)

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
  Promise.resolve().then(() => console.log("C"));  
}, 0);  
  
Promise.resolve().then(() => {  
  console.log("D");  
  setTimeout(() => console.log("E"), 0);  
});  
  
console.log("F");
```

👉Output

```
A // sync  
F // sync  
D // microtask after sync  
B // macrotask (first setTimeout)  
C // microtask scheduled inside B's callback  
E // macrotask scheduled inside D's callback
```



Interview Tips

- Always walk through **sync** → **microtasks** → **macrotasks**.
- Explain with **why** each output happens, not just the result.
- Use phrases like *"this goes to microtask queue, so it runs before timeout"*.
- Shows depth, not just memorization.